CSE 3200: System Development Project

MEDFLOW: AN INTEGRATED OUTPATIENT CARE COORDINATION SYSTEM

By

Apu Sharma
Roll: 2107005

&

Md. Eftakhar Jaman Arfan
Roll: 2107030

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

April, 2026

MedFlow: An Integrated Outpatient Care Coordination System

By

Apu Sharma

Roll: 2107005

&

Md. Eftakar Jaman Arfan

Roll: 2107030

Supervisor:

Dr. K. M. Azharul Hasan

Professor

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

April, 2026

Acknowledgment

All the praise to Almighty Allah, whose blessings and mercy enabled us to complete this project successfully. We express our profound gratitude to our supervisor, Dr. K. M. Azharul Hasan, Professor, Department of Computer Science and Engineering, Khulna University of Engineering & Technology, for his invaluable guidance, constant encouragement, and constructive feedback throughout the course of this work.

We also extend our sincere thanks to the faculty members of the Department of Computer Science and Engineering at KUET for their continuous support and for equipping us with the academic foundation required to undertake a project of this scope. We are grateful to the open-source communities behind NestJS, Prisma, React, and Flutter, whose documentation and tooling made this work possible. Finally, we thank our families and friends for their unwavering moral support throughout the entire duration of this project.

Authors

Abstract

The present healthcare delivery system in Bangladesh comprises of an inefficient booking of appointments, fragmented clinical data, and ineffective coordination of physicians with pharmaceutical suppliers and diagnostic centres, and real-time communication among care providers; this project addresses this gap with **MedFlow: An Integrated Outpatient Care Coordination System** deployed as a NestJS-Prisma-PostgreSQL backend, a React web dashboard, and This system has 5 roles (Patient, Doctor, Pharmacy, Diagnostic, and Admin), role-scoped JWT/RBAC security and outpatient operations are modelled by seven state appointment workflow, five state prescription workflow, and four state lab-order workflow. Web Socket is included in it as well.Real-time notifications at the IO level, processing of prescription and diagnostic documents by Cloudinary, managing profiles, password-reset features, and audit logs in a manner that any notable clinical activity can be tracked and reported to the necessary user. Doctors, pharmacies and diagnostic centres can use role-specific dashboards to organize appointments, prescriptions, reports, notifications and profile data on the web platform and use the Flutter mobile application to cover the patient-facing workflow of doctor browsing, appointment-booking, accessing medical-records, viewing prescription, downloading reports and of receiving notifications. The system had a ten week iterative process of staged analysis, design, implementation and testing process and had automated tests in the backend, web and mobile codebase and had been tested using scenario-driven functional validation of the core clinical workflows. The prototype resulting from it suggests that a multi-platform, software design with type safety and modular features could be a valid and scalable framework to facilitate integrated outpatient care coordination and reduce administrative overhead, improve status visibility, and increase the communication between patients and service providers.

Contents

List of Tables

List of Figures

CHAPTER I

Introduction

1.1 Introduction

Healthcare is becoming more and more digitised in terms of appointment management, prescriptions. diagnostic reports, and communication with the stakeholders to minimize wait time and administrative. error [1, 2]. Still, in Bangladesh, numerous outpatient services rely on. telephone reservations, paper orders and hand coordination between doctors, pharmacies, and diagnostic laboratories.

MedFlow fills this gap as an integrated care coordination system outpatient built. A NestJS backend, a React web application and a Flutter mobile application. It offers role-scoped interfaces between patients, doctors, pharmacies, diagnostic. laboratories and administrators via a single API, and via a WebSocket notification. layer [3]. Combination of appointment, prescription, is its primary contribution. In a single, coherent workflow, lab-order, document-storage, profile, notification and audit workflows. codebase.

1.2 Background / Problem Statement

Outpatient coordination is often based on traditional outpatient coordination through telephone, paper queues. manual result dispatch, handwritten prescriptions, and handwritten prescriptions, and manual result dispatch [4]. This causes duplicate bookings, lost/unclear prescriptions, late result collection, and poor patient. transparency of appointment status.

Current electronic solutions typically address a single aspect of the process. Appointment platforms do not mimic clinical states with confirmation, pharmacy systems do not interface with. Doctors and laboratory systems are not normally connected to appointment records, and laboratory systems. patients. MedFlow thus focuses on the entire process of appointments request and so on. prescription, dispensing, diagnostic reporting and notification.

1.3 Objectives

The principal objectives of this project are as follows.

1. To design and build a system that has role based access , that coordinates patients, doctirs, pharmacy and diagonostic workflow
2. To implement and test the workflow works from appointment to prescribing to lab results and to check for real time notifications
3. To deliver usable multi-platform interfaces (React web and Flutter mobile) for operational workflow execution, status visibility, and document/report access.

1.4 Scope

The project can be divided into three parts : backend , web and mobile. The back-end was built on NestJS v11, Prisma ORM v7, PostgreSQL, Socket.IO, Passport/JWT, bcrypt, PDFKit, Cloudinary, class-validator, and dotenv. The web frontend was built using React 19, TypeScript 5.9, React Router, TanStack Query, Axios, Socket.IO client, Recharts, Zod, Vite, Vitest, and Testing Library. For the mobile app we used Flutter/Dart , GoRouter, `http`, `socket_io_client`, `image_picker`, `permission_handler`, `shared_preferences`, and `url_launcher`. Version control was managed with Git and GitHub.

1.5 Unfamiliarity of the Problem / Topic / Solution

Appointment systems do exist. There are tools for prescription and lab systems as well. But the thing i sthey are all each a separated tools of their own. The combined workflow that we created in medflow is something we did not find in the open-source or academic systems we looked into. [4, 6]. It has a seven state appointment FSM , a five-state FSM for prescription. And there are also the lab workflow , role based websocket notifications and a Fluetter app for the patient side in our overall system. All are integrated in one system keeping in mind the actual workflow of an actual patient.

Table 1.1: RACI Matrix for Project Work Distribution

Task	Apu	Arfan	Supervisor	Reviewer
Requirements Analysis	R/A	C	C	I
DB Schema & Migrations	C	R/A	C	I
Auth & Notifications	R/A	C	I	I
Appointment Module	R/A	C	I	I
Prescription & Labs	C	R/A	I	I
Audit & Cloudinary	C	R/A	I	I
React Web Frontend	C	R/A	I	C
Flutter Mobile App	R/A	C	I	C
Testing	R	R/A	I	C
Report Writing	R/A	R/A	C	C

1.6 Project Planning and Work Distribution

We built the project over ten weeks. Both the members contributed throughout the stack of the sytem. The responsibilities have been shown in a RACI matrix in (Table 1.1) and the overall timeline of the project has been shown in the grant chart (Figure 1.1). Apu Sharma focused on building the backend up from scratch, user level authentication, notification system and the mobile flutter development. MD. Eftakar jaman AArfan designed the schema workflow , the modules , role-specific frontend, the web flow and testing.

R = Responsible, A = Accountable, C = Consulted, I = Informed

1.7 Applications of the Work

MedFlow can be implemented in government outpatient departments, in private clinics, pharmacies. and diagnostic laboratories. It substitutes physical queues with online status monitoring, enables physicians to schedule appointments and records, allows pharmacies to accept prescriptions. data, and allows diagnostic centres to post reports and inform patients. Its REST Audit trail, API, and typed WebSocket events also form a basis of more comprehensive. outpatient interoperability.

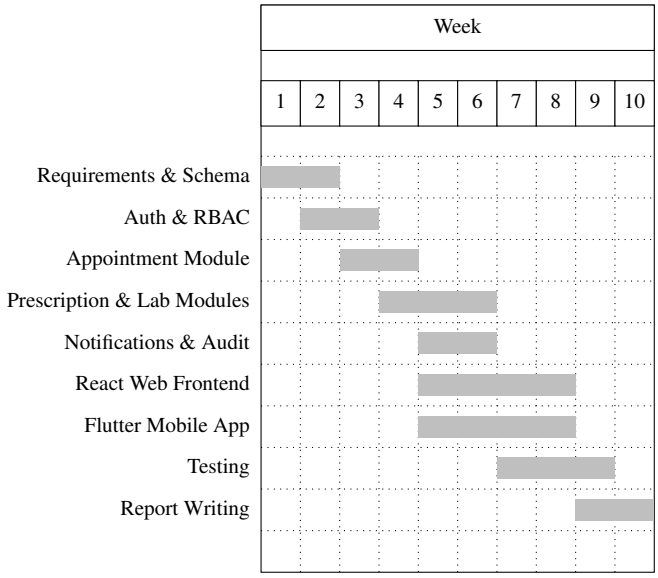

Figure 1.1: Gantt Chart of Project Timeline

1.8 Organization of the Project

Chapter II is an overview of the literature on outpatient care coordination, EHR platforms. pharmacy-laboratory integration, and appointment management. The Chapter III describes the system architecture, database schema and design decisions throughout. all three codebases. Implementation and results are given in Chapter IV. Chapter V answers societal, ethical, legal and environmental issues. Chapter VI analyses the complicated engineering issues and tasks that are faced. Chapter VII concludes the Summary, limitations and recommendations of future work.

CHAPTER II

Related Works

2.1 Introduction

Outpatient care includes patients, physicians, pharmacies, diagnostic labs and. administrative staff. Previous projects have involved big EHR systems to simple booking. mobile health apps and devices. This chapter puts MedFlow in that. landscape as a doctor-patient-pharmacy-laboratory coordination system.

2.2 Related Works

2.2.1 Electronic Health Record Systems

OpenMRS [7] is an open-source EHR system that is widely implemented, yet its appointment. module can only support the Scheduled, Completed and Cancelled states, which is too coarse to support. queue-aware outpatient coordination. OpenEMR [8] incorporates billing and prescribing, although. has poor mobile support and lacks real-time WebSocket notification infrastructure. No system is carrying out the entire loop of the MedFlow: appointment state tracking, prescription forwarding, diagnostic order/result processing, patient mobile notification, and audit logging by a single API.

2.2.2 Commercial Appointment APIs

Calendly and Acuity Scheduling offer general-purpose booking APIs, which are constrained. to hostinvitee scheduling and absence of pharmacy, diagnostic, prescription, lab-result, and intermediate concepts of clinical state [9].

2.2.3 Pharmacy and Laboratory Information Systems

Pharmacy and Laboratory Information System Pharmacy and laboratory Information Systems are the two sub-headings in the category of Information Systems.

Standalone pharmacy systems emphasize inventory, billing and dispensing, and laboratory. systems are oriented on registration of tests, management of samples, and delivery of reports. They are useful within a single organisation, although in most cases are not presented with prescriptions or lab generated by doctors. orders on shared outpatient workflow, and neither do they give coherent updates of patients.

2.2.4 Clinical Workflow Backends

JWT is often implemented by reviewed GitHub repositories and academic theses [10] in general. CRUD authentication and basic appointment. None pair up an appointment FSM, prescription. and laboratories-order workflows and cloud document storage, WebSocket push notifications, and both. Single API react and Flutter clients.

2.2.5 Flutter Healthcare Applications

Flutter healthcare apps are typically concerned with the process of booking appointments or teleconsultation. [11]. Reviewed examples either are based on Firebase or centered on the role of the patient, and without modeling pharmacy and diagnostic stakeholder journeys.

2.3 Discussion

The review systems are compared in Table 2.1. on our implementation along the key differentiating dimensions.

The comparison proves that MedFlow deals with a mix of outpatient coordination. Needs not addressed in any of the reviewed solutions, justifying the novelty of the statement of the problem and its application.

Table 2.1: Comparison of Related Outpatient Coordination Systems

System	**Roles**	**Appt. States**	**Rx Flow**	**Real-time**	**Mobile App**
OpenMRS	4	3	No	No	No
OpenEMR	3	3	Partial	No	No
Calendly API	2	2	No	Webhook	No
Reviewed Backend	2	3	No	No	No
Reviewed Flutter	1	2	No	Firebase	Patient only
MedFlow	**5**	**7**	**5-state**	**WebSocket**	**Flutter (Android+iOS)**

CHAPTER III

System Analysis and Design

3.1 Introduction

The chapter outlines the methodology, architecture, database schema and design. choices between the three codebases. The development was an iterative process. requirements and schema design to backend, frontend, testing, and documentation.

3.2 System Analysis and Design Approach

The project had a five-stage cyclical process.

1. **Phase 1 Requirements and Schema Design:** Stakeholder analysis, role. identification, FSM design, and Prisma schema definition and a migration. strategy planned with fifteen incremental migrations.
2. **Phase 2 - Backend Core:** NestJS module boilerplate, authentication, RBAC guards, appointment service and controller.
3. **Phase 3 - Backend Features:** Prescription, lab order, notification, audit, Cloudinary and profile modules.
4. **Phase 4- Frontend Development:** React web dashboard and Flutter. Parallel mobile application against the stable API.
5. **Phase 5 - Testing and Documentation:** Backend service/controller. tests, web component tests, Flutter unit/widget tests, and report writing.

3.3 Overall Framework and Architecture

In general, its architecture and framework are presented as follows:

Figure 3.1 shows the overall system architecture and is turned over to. preserve readability.

The system is based on three-tier architecture. The **Presentation Tier** holds: the React web SPA and Flutter mobile app, both based on REST and Socket.IO. The **Application Tier** This is the NestJS routing, authentication and business server. logic, FSM enforcement, PDF generation, file upload coordination. The **Data Tier** relies on PostgreSQL using Prisma and Cloudinary avatars, prescription PDFs, and. lab result files.

3.3.1 Data Flow Diagram

The data flow diagram at level-0 of MedFlow is given in Figure 3.2. communication between the external actors and the overall MedFlow system boundary.

The data flow diagram at level-1 of MedFlow is given in Figure 3.3. communication between external actors, key application processes, long-lasting data stores, and email/external file services.

3.3.2 Backend Module Structure

The NestJS application is structured to have fourteen feature modules each having a distinct encapsulation. service and DTOs of its own. These modules are summarised in Table 3.1. and their responsibilities.

3.3.3 Web Frontend Architecture

The React web application is based on role-scoped feature folders, `TanStack Query`, guarded nested routes. and a Socket.Notification event notification client. Recharts powers the Weekly patient-growth and appointment charts by doctor dashboard.

3.3.4 Mobile Frontend Architecture

The Flutter application is based on a layered architecture: its `core/api` layer offers typed REST and `core/domain`, multipart calls, define Dart models and enums. features per

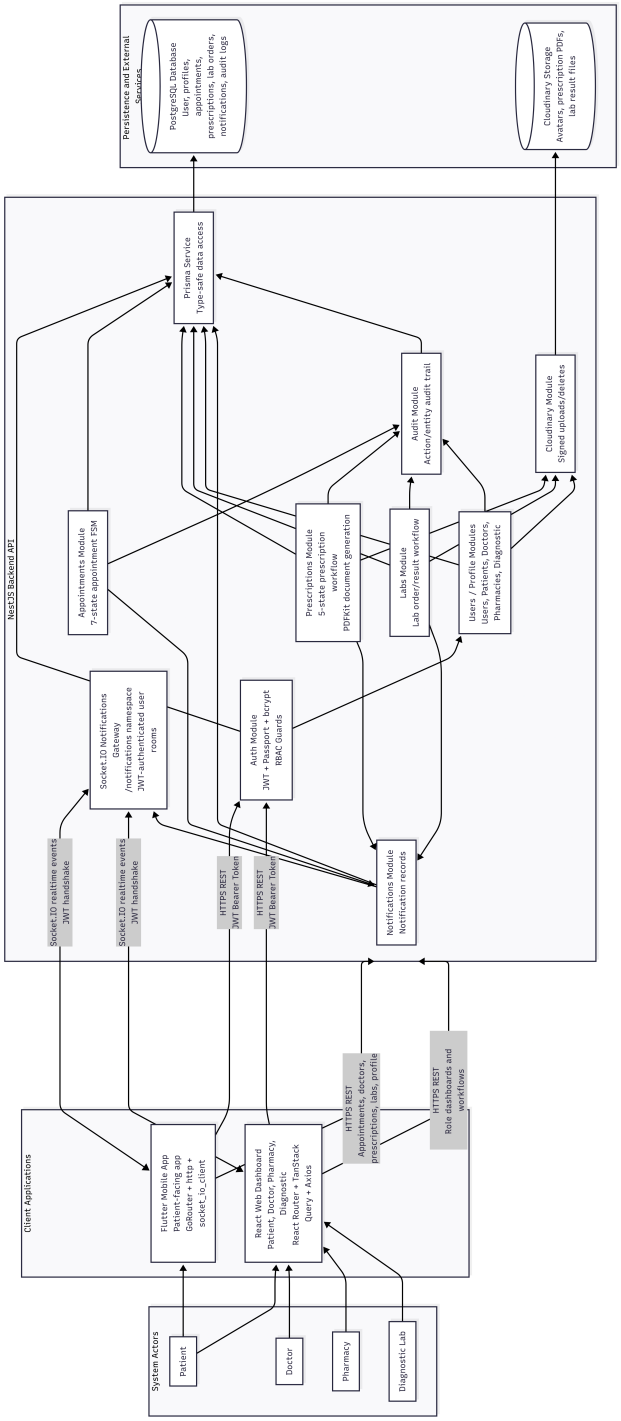

Figure 3.1: Overall System Architecture of MedFlow

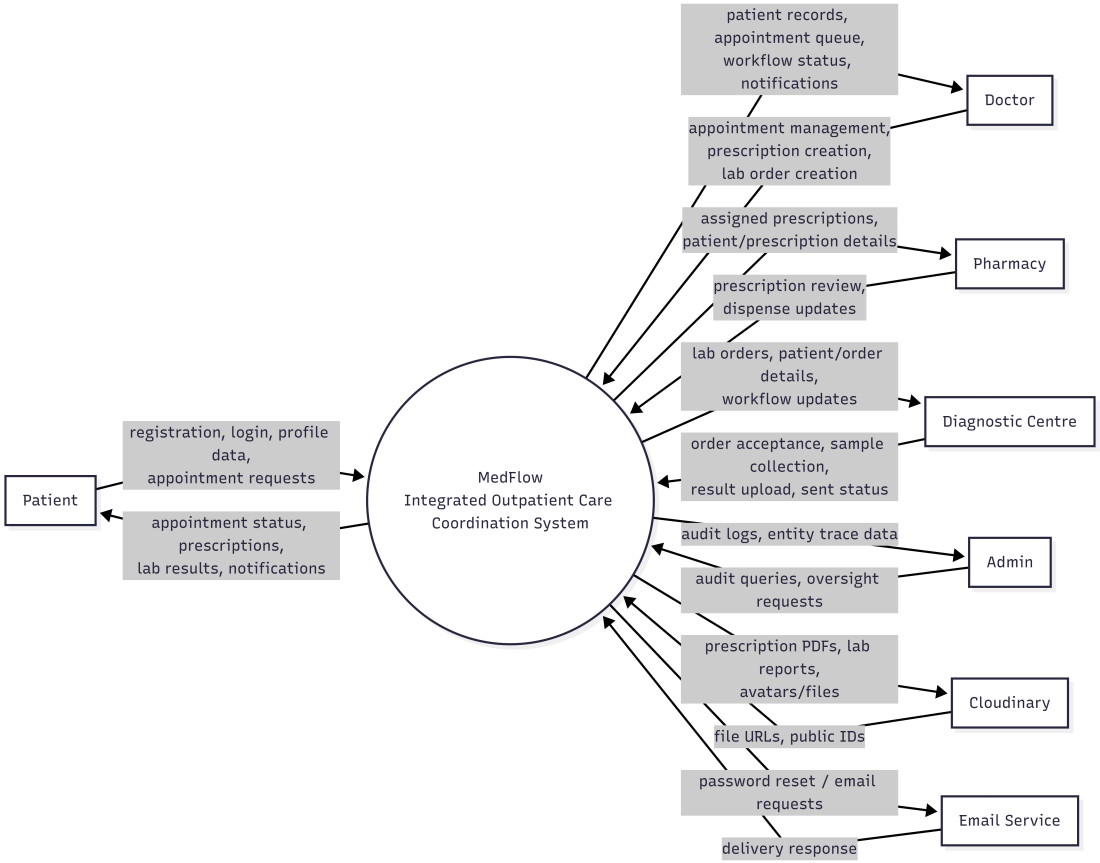

Figure 3.2: Level-0 Data Flow Diagram of MedFlow

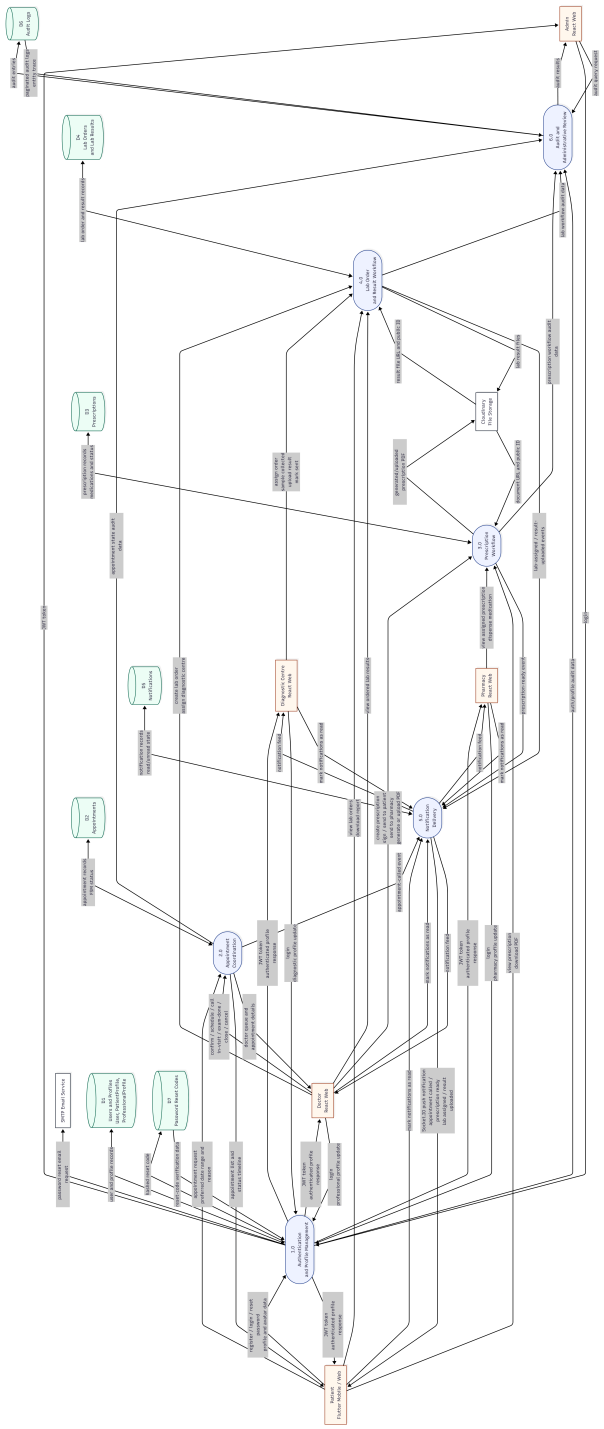

Figure 3.3: Level-1 Data Flow Diagram of MedFlow

Table 3.1: NestJS Backend Module Structure

Module	**Responsibility**
PrismaModule	Wraps PrismaClient as a globally injectable singleton service.
CloudinaryModule	Wraps Cloudinary upload API; provides base64 fallback in development.
HealthModule	Exposes a `GET /health` liveness endpoint.
AuthModule	Registration, login, forgot-password, reset-password, JWT issuance, JwtAuthGuard, RolesGuard.
UsersModule	User lookup and admin-facing user management endpoints.
PatientsModule	Patient profile CRUD, doctor search, and avatar upload.
DoctorsModule	Doctor profile CRUD, public profile viewing, and available time slot management.
PharmaciesModule	Pharmacy profile CRUD.
DiagnosticModule	Diagnostic centre profile CRUD and accreditation management.
AppointmentsModule	Appointment creation, listing, and FSM state transitions; integrates notification and audit services.
PrescriptionsModule	Prescription draft creation, PDF generation, status transitions, and pharmacy assignment.
LabsModule	Lab order creation, assignment, sample collection, and result file upload.
NotificationsModule	Notification creation, listing, mark-read; Socket.IO WebSocket gateway for real-time push.
AuditModule	Audit log recording and admin-facing paginated query endpoint.

patient separates repositories, mappers, controllers, and pages. `GoRouter` is declarative navigation based on a persistent `PatientShell`. `Session Controller` is the controller that powers redirects on login. Live updates come in the form of. `Computersocketio client` and update the notification centre.

3.4 Problem Design and Analysis

This section includes the problem design and analysis.

3.4.1 Role Identification

Outpatient workflow analysis revealed five types of actors.

- **Patient —** Signs up with a medical profile, searches the available physicians, books. appointment requests, check appointment status, prescription and lab views. receives real-time push notifications, results.
- **Doctor —** Handles an appointment queue, promotion of appointment states, views. prescribes, assigns lab, patient medical histories and clinical records. sends requests to diagnostic centres, and views an analytics dashboard.
- **Pharmacy —** Receives prescriptions given by doctors, reads medication de- tails, retrieves prescription PDFs and progresses prescription status until. dispensing.
- **Diagnostic —** Takes lab orders assigned by doctors, accepts orders, gathers samples, uploads result files on Cloudinary and informing patients when. reports are ready.
- **Admin —** Can access paginated logs and has system-wide administrator privileges.

The MedFlow use case diagram is subdivided into four sections that can be read. Its presentation (Figures 3.4 through 3.7 since) is too large to be included within the entire model. single A4 page.

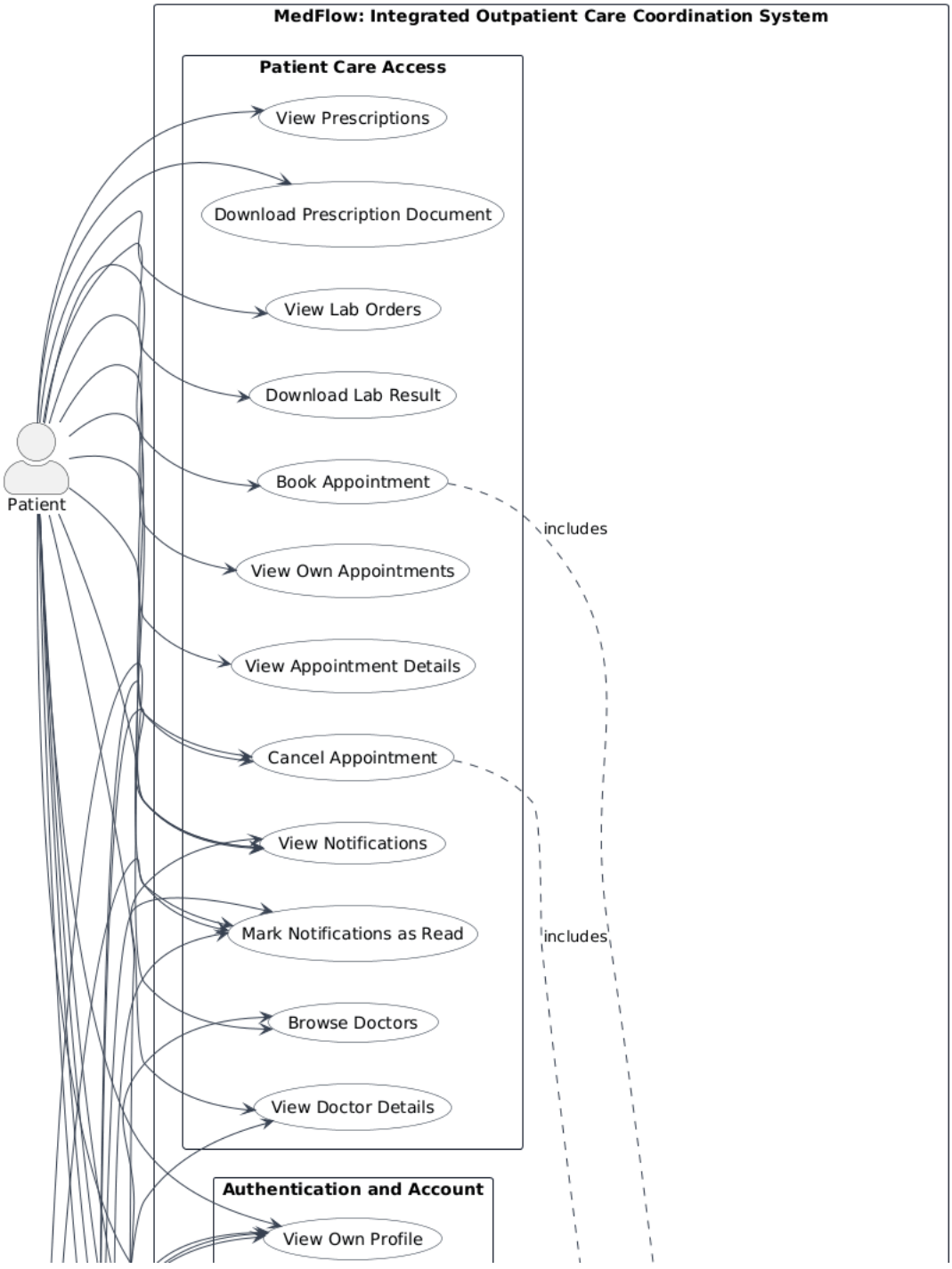

Figure 3.4: Use Case Diagram of MedFlow — Authentication and Patient Care Access

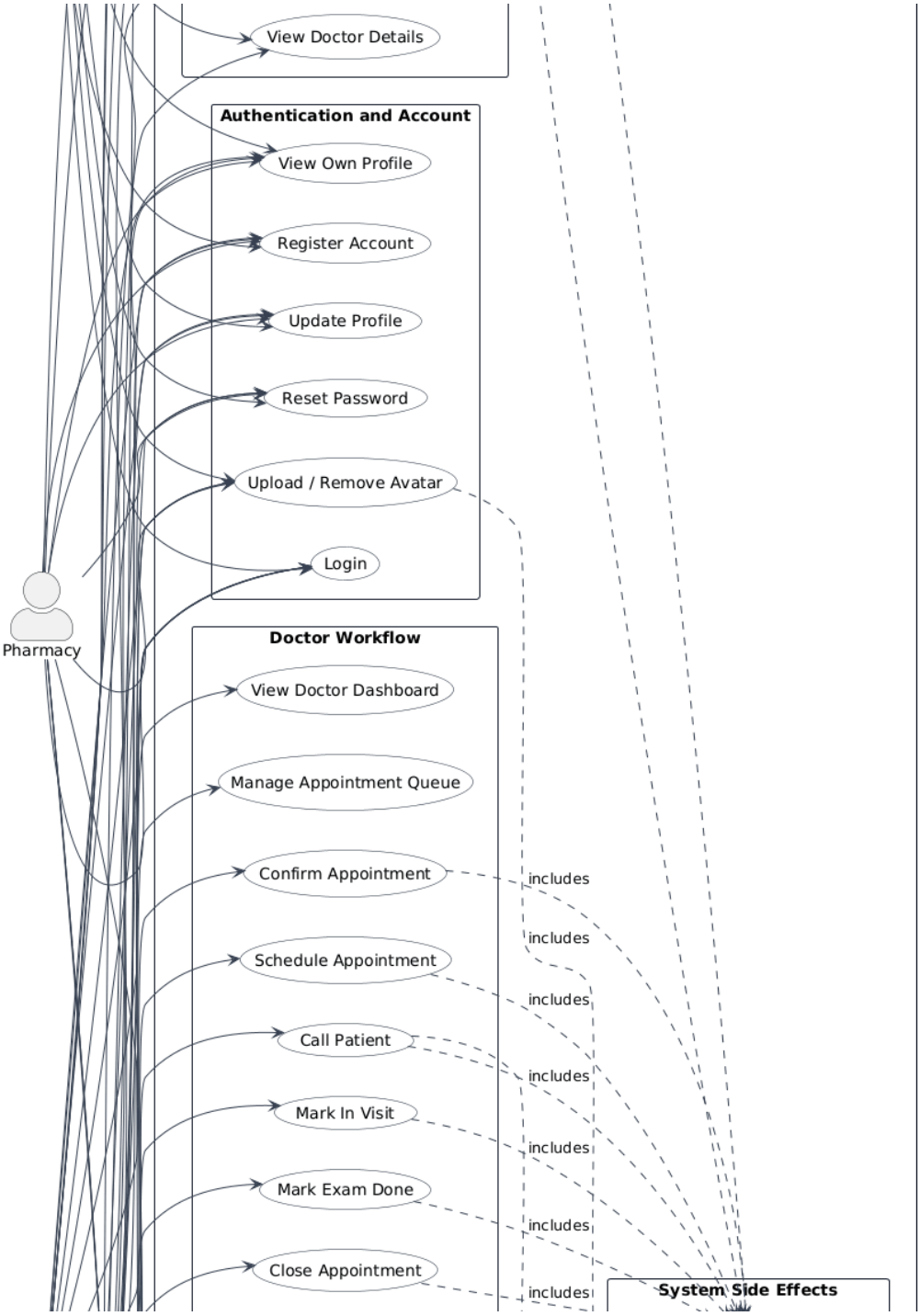

Figure 3.5: Use Case Diagram of MedFlow — Patient and Doctor Workflows

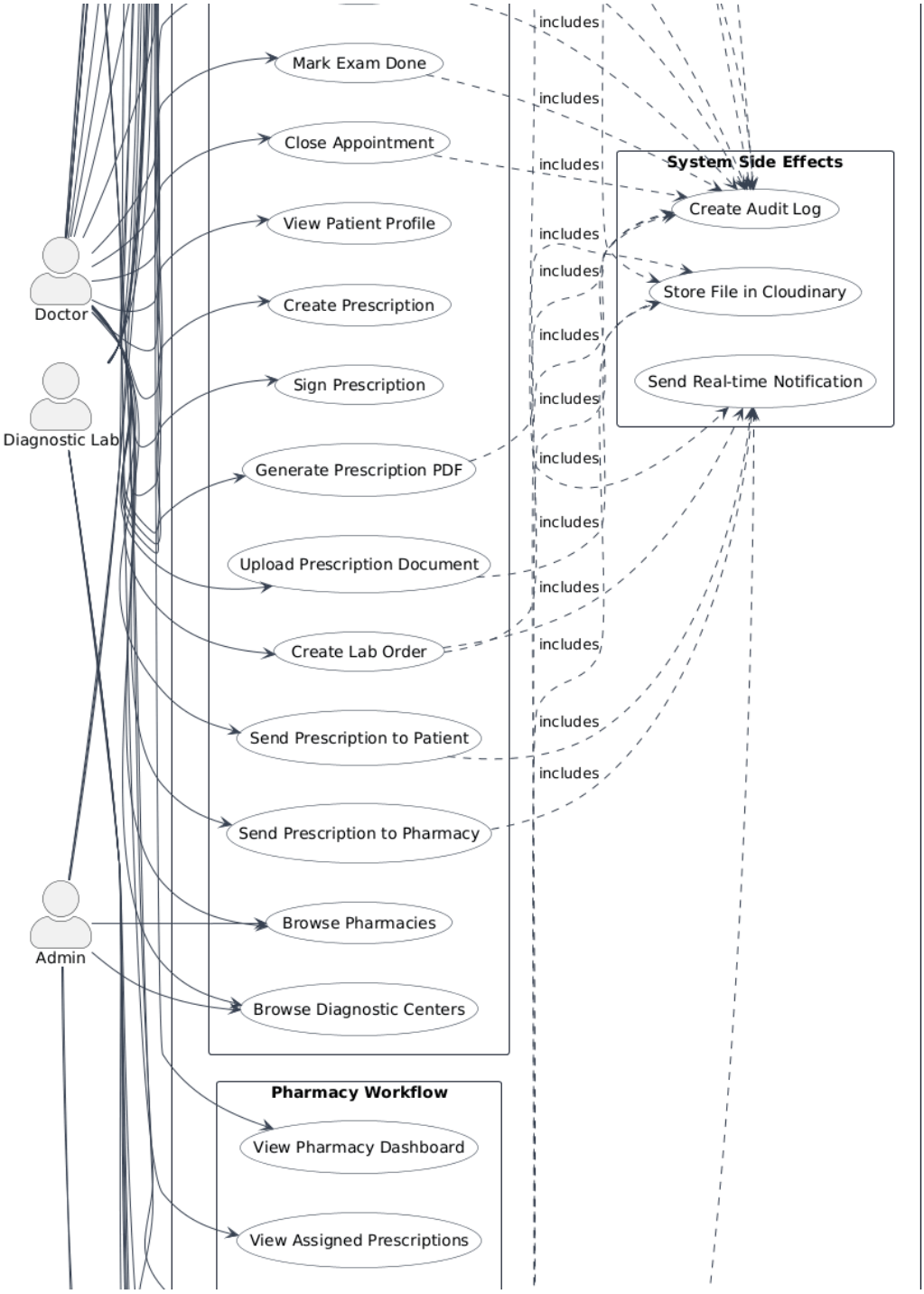

Figure 3.6: Use Case Diagram of MedFlow — Pharmacy and Diagnostic Workflows

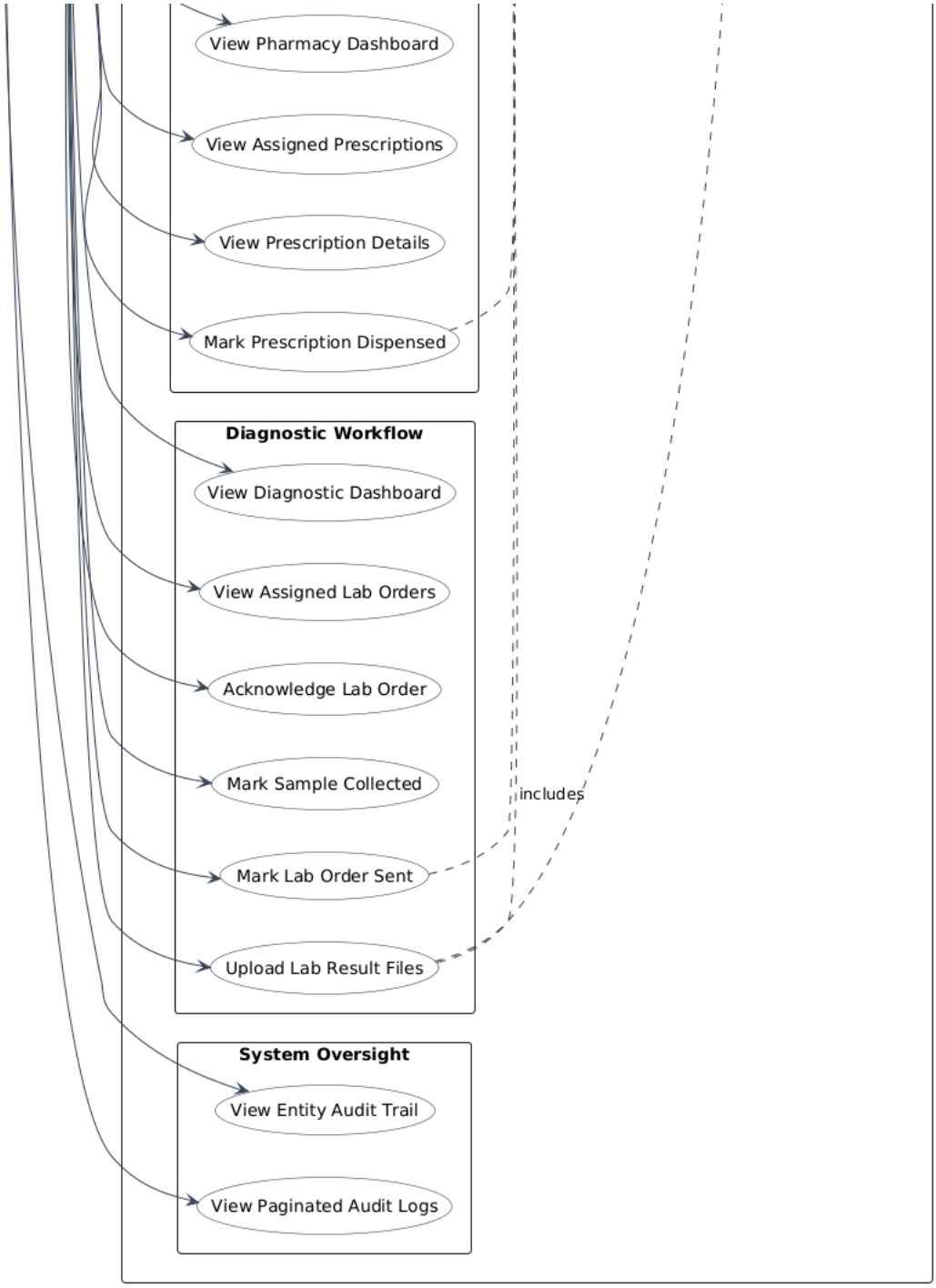

Figure 3.7: Use Case Diagram of MedFlow — Oversight and System Side Effects

Table 3.2: Appointment States and Clinical Semantics

State	Semantics
REQUESTED	Patient has submitted a request; awaiting doctor confirmation. A preferred date range and a time note may accompany the request.
CONFIRMED	Doctor has accepted and the appointment is scheduled.
CALLED	Patient has been called to the waiting area. A push notification is dispatched to the patient at this transition.
IN_VISIT	Patient is currently in the consultation room.
EXAM_DONE	Consultation is complete. Lab orders and prescriptions may now be created.
CLOSED	All associated steps are resolved. Appointment is finalised.
CANCELLED	Appointment was cancelled by patient, doctor, or admin.

Table 3.3: Prescription States and Semantics

State	Semantics
DRAFT	Doctor has created the prescription but not yet finalised it.
SIGNED	Doctor has signed the prescription; the generated PDF document can then be uploaded to Cloudinary.
SENT_TO_PATIENT	Prescription is visible to the patient; a push notification is dispatched.
SENT_TO_PHARMACY	Prescription has been forwarded to the assigned pharmacy.
DISPENSED	Pharmacy has dispensed the medication; the workflow is complete.

3.4.2 Appointment State Machine

The role capability lifecycle was represented by a seven state finite-state machine. Ta-Each state is defined in ble 3.2 and Figure 3.8 shows the allowed transitions as encoded in. the program code on the back-end server.

3.4.3 Prescription State Machine

Prescriptions are an independent five-state lifecycle also reference by a transition. map of the prescriptions service. Each state is defined in Table 3.3.

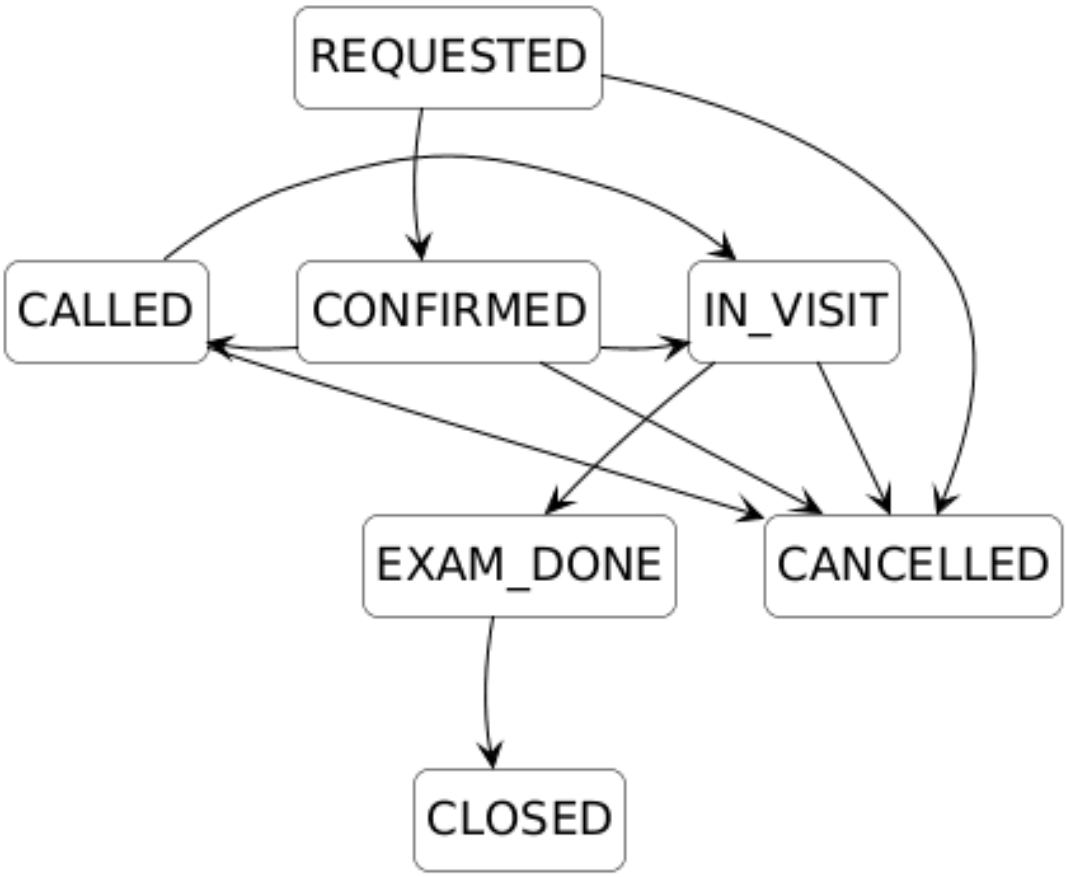

Figure 3.8: Appointment Finite-State Machine

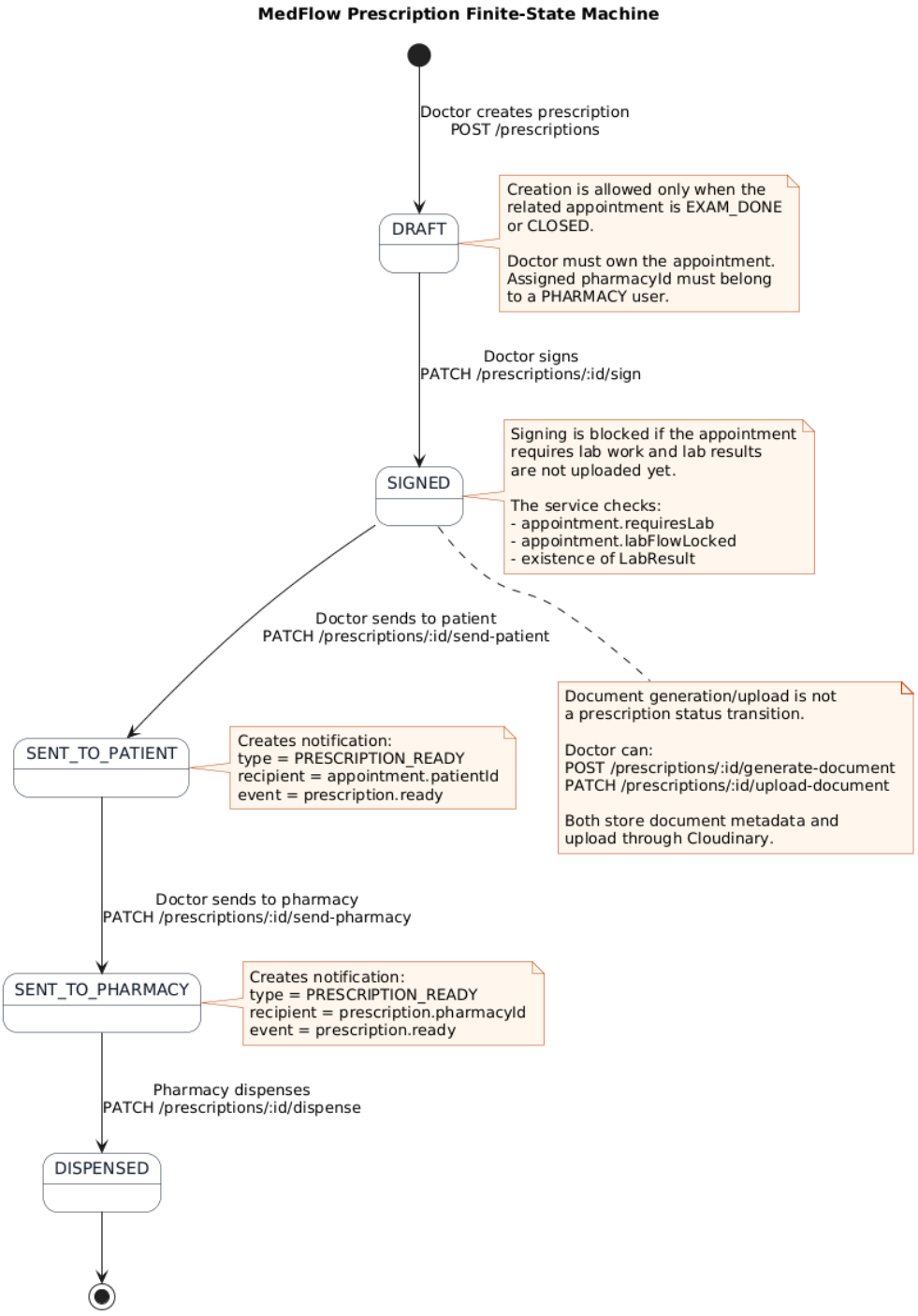

Figure 3.9: Prescription Finite-State Machine

Table 3.4: Lab Order States and Semantics

State	Semantics
CREATED	Doctor has created the order and assigned a diagnostic centre.
ASSIGNED	Diagnostic centre has acknowledged the order; push notification dispatched to patient.
SAMPLE_ COLLECTED	Diagnostic centre has collected the biological sample.
SENT	Result file uploaded to Cloudinary; push notification dispatched; patient may now download the report.

3.4.4 Lab Order State Machine

Orders in Labs have a four-state lifecycle which is categorized in Table 3.4.

3.5 Database Schema Design

There are ten Prisma schema models. An overall projection of is given in Table 3.5. if it has a model, its essential fields, and relationships.

The entity relationships that were produced based on the real Prisma schema are displayed in Figure 3.11.

3.6 Conclusion

The decision-making process of the system analysis and design remained based on outpatient work-. flows. Prisma is the source of truth on the data-layer, and NestJS modules are modules on the backend. concerns, and the web and mobile clients have equal API contract via TypeScript. and Dart models.

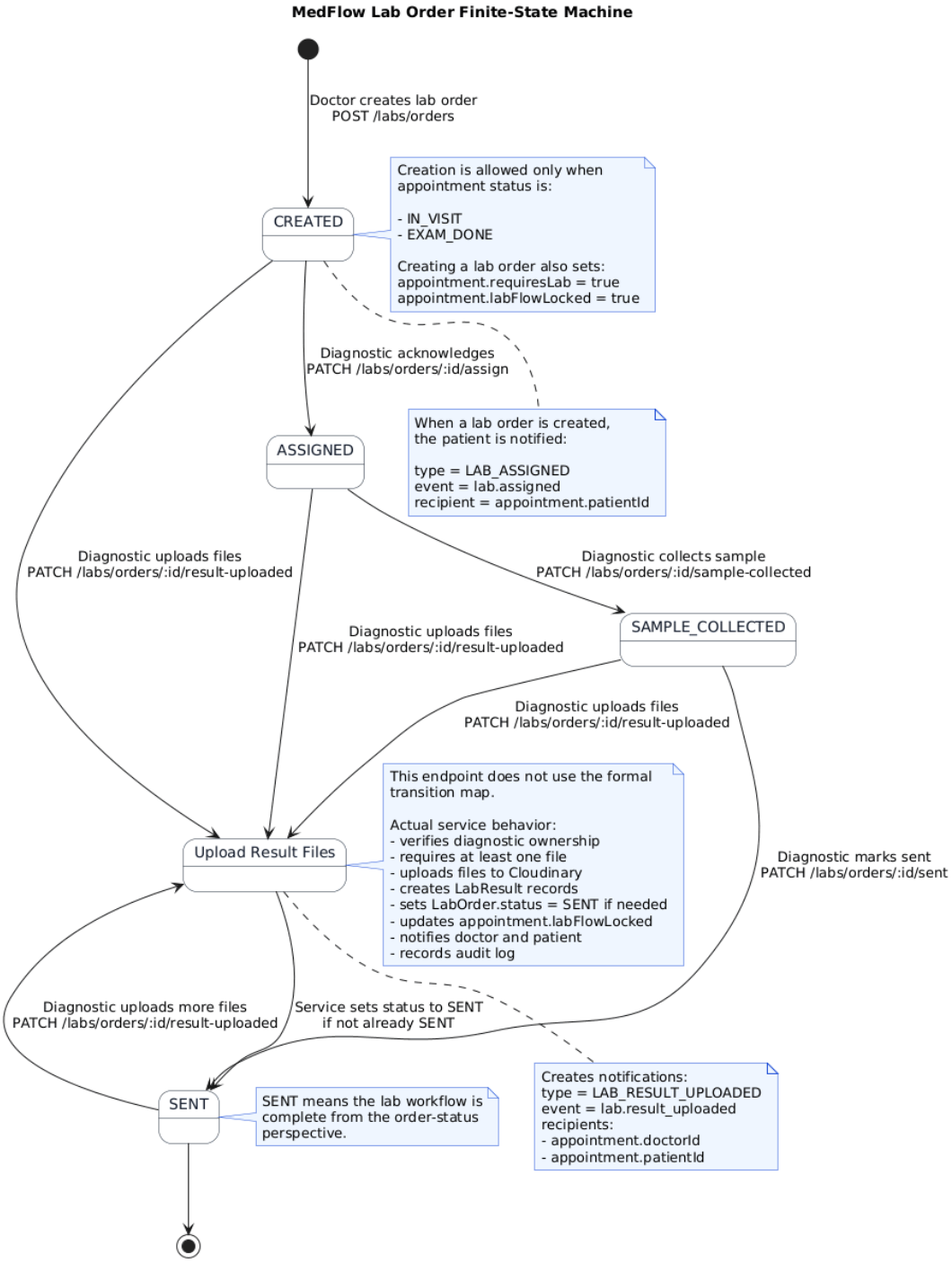

Figure 3.10: Lab Order Finite-State Machine

Table 3.5: Database Schema — Model Overview

Model	**Key Fields and Relationships**
User	UUID PK, fullName, avatarUrl, email (unique), passwordHash, role (enum), phone, address; relates to PatientProfile, ProfessionalProfile, Appointments (as patient and doctor), LabOrders (as diagnostic), Prescriptions (as doctor and pharmacy), Notifications, AuditLogs, PasswordResetCodes.
PatientProfile	One-to-one with User; stores dateOfBirth, gender, allergies, chronicConditions, currentMedications, and emergency contact fields.
ProfessionalProfile	One-to-one with User; stores licenseNumber, specialization, about, clinicName, clinicAddress, availableTimeSlots (JSON), degrees (JSON), certifications (JSON), yearsOfExperience, availableTests (JSON); fields used selectively based on role.
Appointment	UUID PK; patientId and doctorId FKs to User; status (AppointmentStatus enum); scheduledAt, reason, preferredDateFrom, preferredDateTo, preferredTimeNote, requiresLab, labFlowLocked; relates to LabOrders and Prescriptions.
LabOrder	UUID PK; appointmentId FK; diagnosticId FK to User; status (LabOrderStatus enum); tests (JSON array); relates to LabResults.
LabResult	UUID PK; labOrderId FK; fileUrl, filePublicId, fileMimeType, fileSizeBytes; uploadedAt timestamp.
Prescription	UUID PK; appointmentId FK; doctorId and pharmacyId FKs to User; notes, diagnosis, instructions, medications (JSON); status (PrescriptionStatus enum); documentUrl, documentPublicId, documentVersion.
Notification	UUID PK; userId FK; type (NotificationType enum: APPOINTMENT_CALLED, LAB_ASSIGNED, LAB_RESULT_UPLOADED, PRESCRIPTION_READY); message; read boolean.
AuditLog	UUID PK; actorUserId FK; action, entityType, entityId strings; metadata (JSON); createdAt.
PasswordResetCode	UUID PK; userId FK; email; codeHash (SHA-256); expiresAt; consumedAt; attemptCount; rate-limited to 5 attempts.

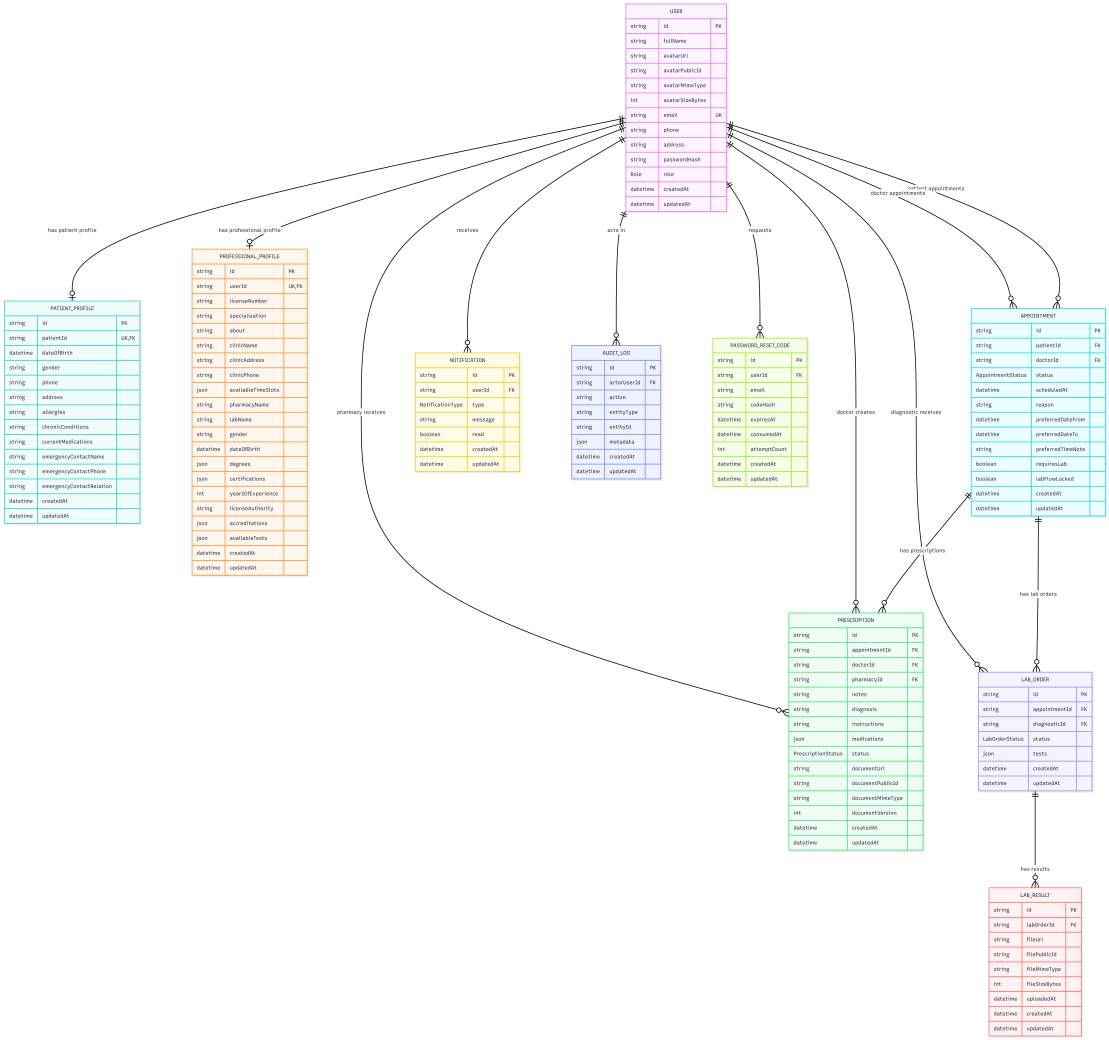

Figure 3.11: Entity Relationship Diagram of MedFlow

CHAPTER IV

Implementation, Results and Discussions

4.1 Introduction

The chapter includes implementation, API design, the frontend structure, and test. outcomes within the three components of the applications.

4.2 Experimental Setup

Components were created and tested on Ubuntu 22.04 LTS with Node.js. v20, npm 10, NestJS CLI 11, Prisma CLI 7, PostgreSQL in Docker, TypeScript 5.9, Flutter/Dart, Vite, Vitest with Testing Library, and Flutter Test: 3.11.

4.3 Evaluation Metrics

Assessment included correctness of backend services/controllers, and role enforcement, FSM. integrity, dispatches notification to asserted rooms and schema-level data integrity and role-gated page behaviour, loading behaviour, and error behaviour.

4.4 Dataset

Synthetic test data has been developed using `backend/prisma/seed.ts`. It includes Product features (and URLs) patient and professional profiles, appointments, prescriptions, lab orders, lab results, and notifications, giving a realistic foundation of manual and automated. testing.

4.5 Implementation and Results

4.5.1 Backend Implementation

Authentication and Registration

Registration takes one role specific profile data request. Patients submit medical although professional profile is submitted by doctors, pharmacies, and diagnostic users, profile fields. fields. `bcrypt` is used to hash passwords, and the login provides a JWT with user UUID, email, and role. A six-digit code that is stored in the password reset flow as a hash of `SHA-256`. protecting expiry, attempt limiting and consumed state.

Appointment Service

The appointment service puts a list of allowed transitions in a look up table (Listing IV.1). Trigger transitions, like `CALLED`, do not reset the state, but send out a `Socket.IO` cast to authenticated room of patient.

```
const TRANSITIONS: Record<AppointmentStatus, AppointmentStatus[]> = {
  REQUESTED: [CONFIRMED, CANCELLED],
  CONFIRMED: [CALLED, IN_VISIT, CANCELLED],
  CALLED:    [IN_VISIT, CANCELLED],
  IN_VISIT:  [EXAM_DONE, CANCELLED],
  EXAM_DONE: [CLOSED],
  CLOSED:    [],
  CANCELLED: [],
};
```

Listing IV.1: Appointment FSM Transition Map

Prescription Service

Only when the appointment is `EXAM_DONE` or less, it can create prescriptions. `CLOSED`. Physicians put signatures on prescriptions, create or upload PDF files using Cloudinary, and forwards the prescription FSM. Once a prescription is sent to `SENT_TO_PATIENT`,

a PRESCRIPTION_READY notification is sent.

Lab Order and Result Service

Lab orders are developed by doctors and include a diagnostic centre and requested tests. ASSIGNED and SENT transition informs the patient; uploading of the result means saving the file in Cloudinary and develops a LabResult record.

WebSocket Notification Gateway

All Sockets are authenticated by the NotificationsGateway Socket.IO connection with Join to user: joins it to user:{userId}. Emits: services call emitToUser(userId, event, payload) to only broadcast to that room; spoisy tokens have their connection cut off.

Figure 4.1 illustrates a sequence that was implemented to further advance an appointment to CALLED, authorization, transition validation, notification persistence, Socket.IO emission, and audit logging.

Cloudinary Service

The CloudinaryService encrypts upload requests using HMAC-SHA256 and re-encrypts the uploads. appendages the URL, public ID, version, MIME type and the size of the bytes. Without Cloudinary credentials, it provides a base64 data URI to allow development processes to continue.

4.5.2 REST API Endpoints

The major API groups are summarised in Table 4.1. Specific transition is revealed through controllers. endpoints, rather than a generic endpoint in the status of workflows, where invalid changes of any workflow may be. rejected clearly.

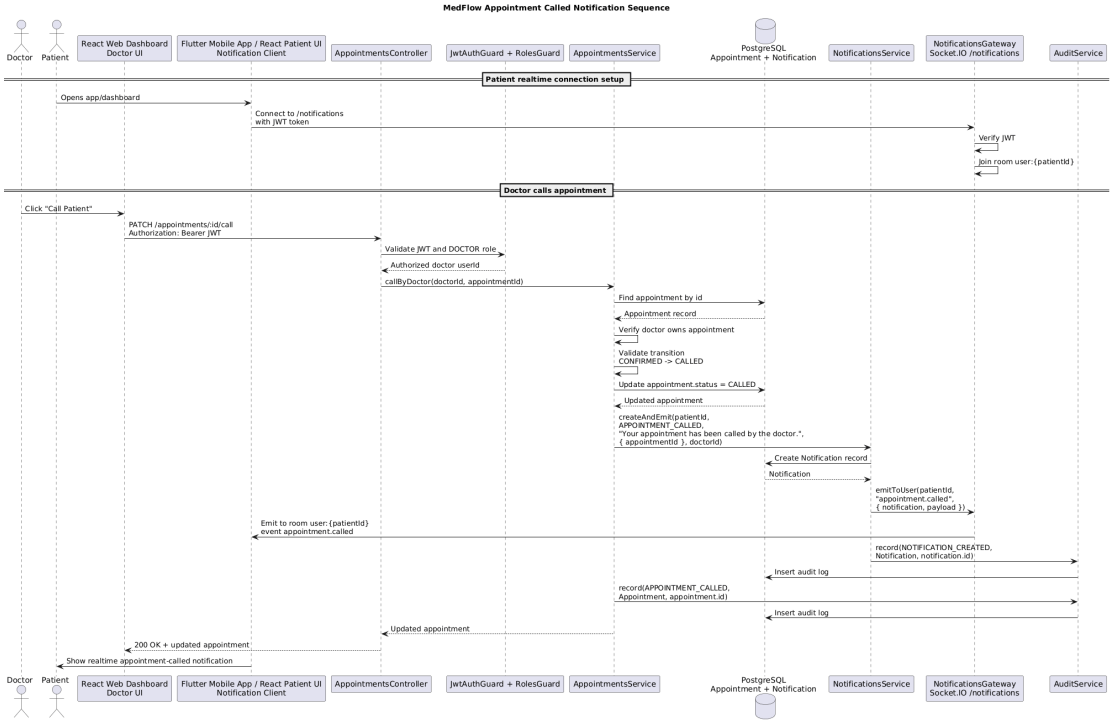

Figure 4.1: Appointment Called Notification Sequence

Table 4.1: Principal REST API Endpoint Groups

Area	**Representative Paths**	**Roles**	**Purpose**
Authentication	/auth/register, /auth/login, /auth/forgot-password, /auth/reset-password	Public	Registration, JWT login, and password reset.
Appointments	/appointments, /appointments/me, /appointments/:id/confirm, /schedule, /call, /in-visit, /exam-done, /close, /cancel	Patient, Doctor	Booking and appointment FSM transitions.
Prescriptions	/prescriptions, /prescriptions/:id/sign, /send-patient, /send-pharmacy, /dispense, /upload-document, /generate-document	Doctor, Pharmacy	Prescription lifecycle, PDF generation, and dispensing.
Lab Orders	/labs/orders, /labs/orders/me, /labs/orders/:id/assign, /sample-collected, /result-uploaded, /sent, /result	Doctor, Diagnostic, Patient	Lab order creation, diagnostic workflow, and result access.
Notifications	/notifications/me, /notifications/me/read	All authenticated users	Notification listing and read-state updates.
Profiles	/users/doctors, /users/me/avatar, /doctors/me/profile	Patient, Doctor, all authenticated users	Doctor browsing and profile/avatar management.
Audit	/audit, /audit/me, /audit/entity/:entityType/:entityId	Admin, authenticated users	Paginated audit review and entity-specific traces.

Table 4.2: React Web Frontend Pages by Role

Role	Pages Implemented
Patient	Appointments list, Appointment details (status timeline, prescription cards, lab order cards), Doctor details, Notifications, Medical records, Profile with avatar upload.
Doctor	Home dashboard (weekly appointment bar chart, patient growth line chart, recent activity feed), Appointments list and detail, Patient list and individual patient profile, Prescriptions, Lab orders, Notifications, Profile.
Pharmacy	Home, Prescriptions list, Prescription detail (medication table, download PDF), Notifications, Profile.
Diagnostic	Home, Lab orders list, Lab order detail (test list, upload result file), Notifications, Profile.

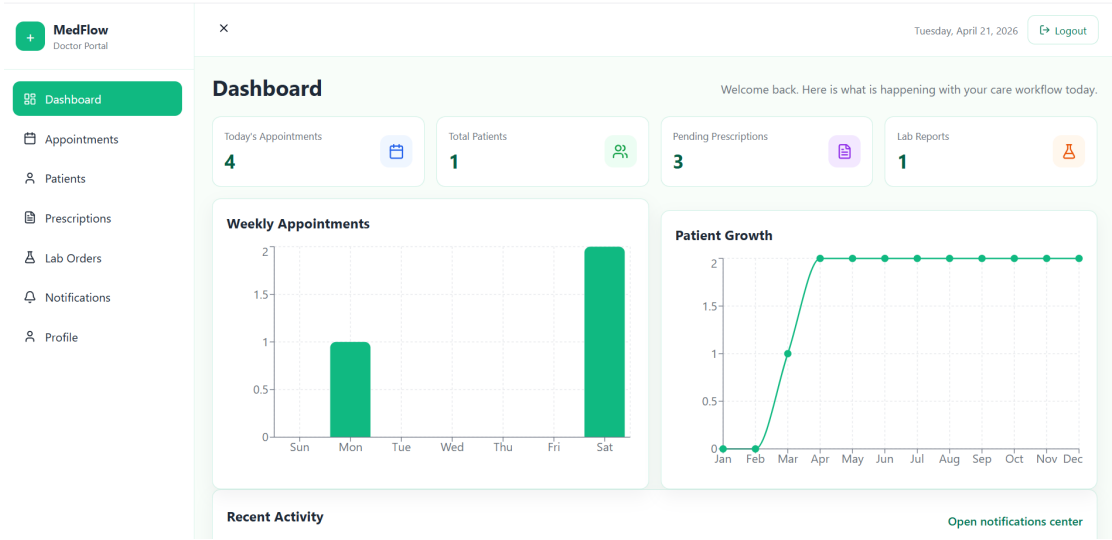

Figure 4.2: Doctor Dashboard and Analytics Interface

4.5.3 React Web Frontend Implementation

The web-based system has specific dashboards to display Patient, Doctor, Pharmacy. and Diagnostic roles. `RoleHomeRedirect` redirects every user to the appropriate home route after going through the login. Table 4.2 is a summary of pages implemented.

Appointments, lab orders, prescriptions and notifications are loaded by Doctor home page. Connects to `TanStack Query`, and visualizes weekly appointment and patient-growth analytics. using Recharts.

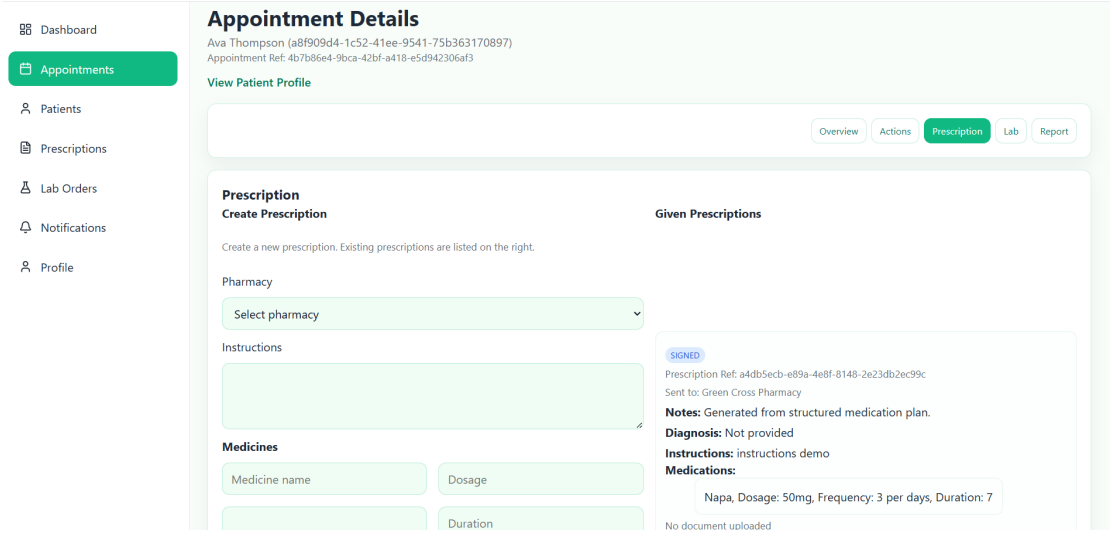

Figure 4.3: Doctor Appointment Details and Prescription Creation Interface

4.5.4 Flutter Mobile Frontend Implementation

The Flutter application has the implication of the patient journey: authentication, doctor brows- ing, making appointments, appointment information, prescription and report downloads, medical. notifications, management of records and profile/avatar. Listo `GoRouter` and `PatientShell` pro- vide guarded navigating and a bottom persistent bar, with the JWT being stored in `SessionController` and redirects being triggered. Real-time messages update the unread number using. `NotificationsRealtimeService` and `NotificationsCenterController`.

4.5.5 Qualitative Results

The React web app, Flutter android emulator, and Thunder Client were tested manually. Role redirect, FSM transitions, notifications, uploads, downloads, and doctor analytics. worked as expected.

4.5.6 Quantitative Results

The results of the three automated test suites are summarised in Table 4.3. bases. The default `npm test` suite is the result of the backend; the backend `test:e2e` is the separate one. this does not include command.

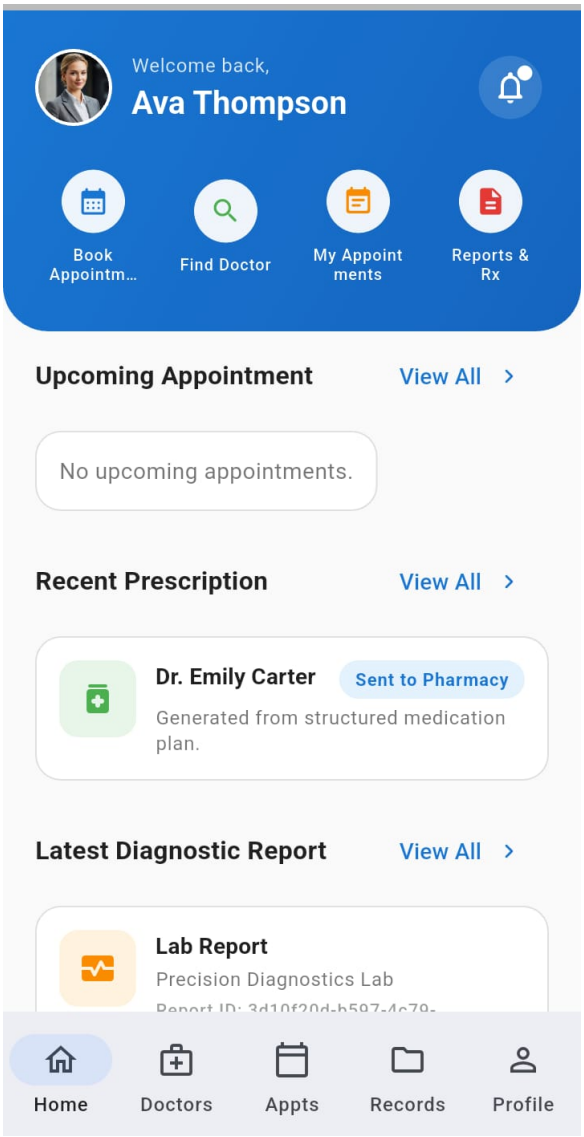

Figure 4.4: Patient Mobile Dashboard Interface

Table 4.3: Automated Test Suite Results

Test Category	**Total**	**Passed**	**Failed**
Backend — Jest unit tests	102	102	0
Web — Vitest unit/component tests	79	79	0
Mobile — Flutter unit/widget tests	75	75	0
Total	**256**	**256**	**0**

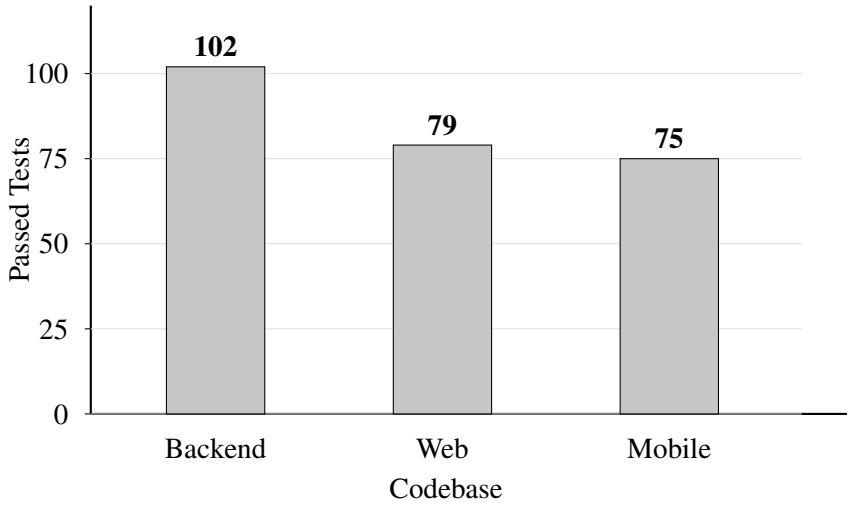

Figure 4.5: Distribution of Passing Automated Tests Across Codebases

The same passing-test distribution is visualised in Figure 4.5; use the table as a source of precise totals.

4.5.7 Objective Achievement

Each of the eight goals in Chapter I was achieved: five-role authentication, appointment, prescription and lab-order FSMs, PDF and Cloudinary workflows, Authenticated WebSocket. React dashboards, role dashboards, and the Flutter patient app as well as audit logging. significant state changes.

4.6 Conclusion

The deployment shows that a NestJS and Prisma backend, with both a(s). Can be react SPA and a Flutter mobile client using a shared WebSocket and REST API. capable of modeling and implementation of the complicated role and lifecycle necessities of a real-world. delivery of outpatient care situation.

CHAPTER V

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

The project is built using the assistance of open-source tools that are free of charge like `NestJS`, `Prisma`, `React`, and so on. `PostgreSQL`, `PDFKit`, `Flutter`, and `Socket.IO`. `Cloudinary` SDK, and IO. No proprietary third-party real patient/code was involved; the codebase of the project is original work by the author.

5.2 Ethical Considerations

The system handles sensitive health records such as medical records, diagnoses, etc. prescriptions, and diagnostic results. It utilises data minimisation, hash passwording using `bcrypt`. audit logging, role based access and secure password reset codes, hashed and expiring, bleeding bounds and state protection.

5.3 Safety Considerations

Input validation and FSM validators are implemented with the help of DTO decorators in order to prevent clin- JWT sends tokens to invalid `Socket.IO`, iconically nonsense states. clients are not connected, `Prisma` (parameterizes database queries) and `Cloudinary` uploads are `HMAC- SHA256` signed.

5.4 Legal Considerations

Production deployment would be required in Bangladesh that would comply with the Digital. Security Act 2018. There would also be the need of functions to deployments of

European Union residents. GDPR-aligned erasure, portability, retention and privacy practices. The full compliance would also involve rest encryption, privacy notice, retention/deletion policy and data. protection assessment.

5.5 Impact of the Project on Societal, Health, and Cultural Issues

Appointment in real-time will help MedFlow to avoid the waiting-room long queues. enhance, decreases losses in prescriptions or misinterpretation by making them digital, and minimises. drive to gather lab report. It also follows the Bangladeshi definition of family involvement by. bringing appointment and result to a personal device.

5.6 Impact of Project on the Environment and Sustainability

The use of paper is reduced by electronic booking, prescriptions and diagnoses. unnecessary travel. Cloud hosting can also reduce idleness in infrastructure pretty much compared with dedicated. on-premise servers, and making healthcare administration less environmentally-intensive.

CHAPTER VI

Addressing Complex Engineering Problems and Activities

6.1 Complex Engineering Problems Associated with the Current Project

Numerous intricacies were addressed. The appointment, prescription and First. FSMs lab ordered were needed to perform transition validation and to do other side effects such as notification. leave This leaves no trace with PDF generation, `Cloudinary` upload and audit logging. Second, the WebSocket gateway was supposed to have secure reconnection behaviour, which was taken care of with the usage of JWT- authenticated per-user rooms. Third, the state of the authentication had to conform in the Flutter application. via navigation and real-time events, processed with a `SessionController` monitored. by `GoRouter`. Fourth, the `Appointment` was a relation that had to be named in `Prisma`. `Appointment` refers To patient and physician.

6.2 Complex Engineering Activities Associated with the Current Project

The project included evolution of a database; 15 `Prisma` migrations. `Cloudinary` `PDFKit` Buffers `PDFKit` Buffers and `Cloudinary` Uploads `PDFKit` PDF pipeline over JSON medication data. stored document URLs, a layered Flutter model with maps, repositories, models, and controllers. `React`/`Vitest` role-flow tests and testing had to be mocked as well by backend `Jest`. `Flutter` unit/widget tests.

CHAPTER VII

Conclusions

7.1 Summary

MedFlow: An Integrated Out-, was designed, implemented and evaluated in this project. Connecting Patient Care. The server in `NestJS`/`Prisma`/`PostgreSQL` is an implementation of. 10 examples, 5 roles, appointment/prescription/lab FSMs, Web Socke notifications, PDF. generation, Cloudiny storage, audit logging and password reset. The `React` web frontend. provides job-specific dashboards and workflows, and the `Flutter` UI provides a mobile experience. the patient experience of scheduling doctor visits, prescription and lab visits. reports. The certified automated suites fulfilled 256 backend, web and mobile tests.

7.2 Limitations

The YouToPhone application now has five major limitations: the mobile application only has. totients; Redis-prohibited sockets and load.IO Scaling has not been done; no certified. OWASP Top 10 security audit has been deployed; email delivery is just a prototype stub; and appointments are not in yet reserving slots on availability of doctors.

7.3 Recommendations and Future Works

The mobile assistance to the doctor, pharmacy and diagnostic functions must be extended into the future, add a `Socket.IO` Redis email adapter, an email production service.Validate, and scale-out. request of appointment in case of unavailability of doctors. `WebRTC` are other extensions. teleconsultation, an Admin audit dashboard, connection with the national patient identifier, ML- no-show/specialist/prescription analysis assistants, formal audit of security and privacy assistant. before actual practice with patients.

References

[1] Bates, D. W. and Gawande, A. A., 2003, "Improving Safety with Information Technology," *New England Journal of Medicine*, Vol. 348, No. 25, pp. 2526–2534.

[2] Shortliffe, E. H. and Cimino, J. J. (Eds.), 2014, *Biomedical Informatics: Computer Applications in Health Care and Biomedicine*, 4th ed., Springer, New York.

[3] Richardson, L. and Ruby, S., 2007, *RESTful Web Services*, O'Reilly Media, Sebastopol, CA.

[4] Hossain, M. S. and Ahmed, F., 2019, "Challenges of Healthcare Information Systems in Bangladesh," *International Journal of Healthcare Information Systems and Informatics*, Vol. 14, No. 2, pp. 1–15.

[5] Kamrul, I. and Rahman, M., 2021, "A Review of Appointment Scheduling Systems for Outpatient Clinics in South Asia," *Journal of Health Informatics in Developing Countries*, Vol. 15, No. 1, pp. 45–60.

[6] Witten, I. H. and Frank, E., 2011, *Data Mining: Practical Machine Learning Tools and Techniques*, 3rd ed., Morgan Kaufmann, Burlington, MA.

[7] Mamlin, B. W. *et al.*, 2006, "Cooking Up an Open-Source EMR for Developing Countries: OpenMRS — A Recipe for Successful Collaboration," *AMIA Annual Symposium Proceedings*, pp. 529–533.

[8] Bowen, W., 2007, "OpenEMR: An Open-Source Electronic Medical Record Application," *Linux Journal*, Vol. 2007, No. 158, pp. 2.

[9] Fielding, R. T., 2000, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, University of California, Irvine.

[10] Kaminsky, D. and Shah, R., 2022, "NestJS in Enterprise Healthcare Backends: A Systematic Review," *IEEE International Conference on Software Engineering*, pp. 112–119.

[11] Bui, T. A. and Nguyen, H. T., 2021, "Flutter-Based Telemedicine and Appointment Applications: Design Patterns and Challenges," *International Journal of Mobile Computing and Multimedia Communications*, Vol. 12, No. 3, pp. 18–34.

[12] Prisma, 2023, “Prisma ORM Documentation,” Available online: https://www.prisma.io/docs, Accessed: December 2023.

[13] NestJS, 2023, “NestJS Documentation,” Available online: https://docs.nestjs.com, Accessed: December 2023.

[14] Flutter, 2023, “Flutter Documentation,” Available online: https://docs.flutter.dev, Accessed: December 2023.